



Linux – le multithreading en C++

Dans cet article, nous allons parler de la plateforme Linux. Nous allons expliciter ce qu'est du multithreading et comment en faire en C++ sous Linux. Pour cela nous allons prendre comme support la STL (Standard Template Library) du C++ qui fournit tout ce qu'il faut.

Un environnement intégré : VSCode

Microsoft fournit un IDE gratuit et léger qui est VSCode. Cet IDE est disponible sous Windows, Mac et Linux. Mon poste est sous Ubuntu et VSCode est sympa à utiliser. **1**

Comme vous pouvez le voir sur la photo d'écran, VSCode contient un terminal pour lancer la compilation.

Linux

Mon environnement est un Ubuntu Desktop 18.04. **2**

Maintenant que nous avons un système Linux et un IDE, il nous reste à choisir un compilateur : GCC. C'est le standard historique. On ne change pas une équipe qui gagne ! Avec un g++ -v, je suis en version GCC 7.3. Il supporte le C++11 et C++14 et C++17 : c'est l'essentiel.

Le multithreading et les threads, c'est quoi ?

Un thread est une routine qui s'exécute en arrière-plan. C'est le système d'exploitation qui gère les threads. Sous Windows, le système d'exploitation ordonnance des threads, sous Linux, le système d'exploitation ordonnance des processus. Le thread est un mécanisme léger. Le thread permet d'accéder à des ressources ; pour éviter les conflits d'accès, il faut protéger ces ressources. Exemple : si une variable globale est utilisée dans deux threads qui s'exécutent,



il peut y avoir une corruption de mémoire. Le premier thread fait une opération d'affectation, il est préempté (il devient inactif) et le deuxième thread devient actif et modifie à son tour la variable. Dans ce cas précis, il y a corruption de mémoire car l'emplacement mémoire est modifié au même moment... Pour éviter cela, on va mettre un bloqueur de type mutex, semaphore ou barrier.

La STL ou les API systèmes ?

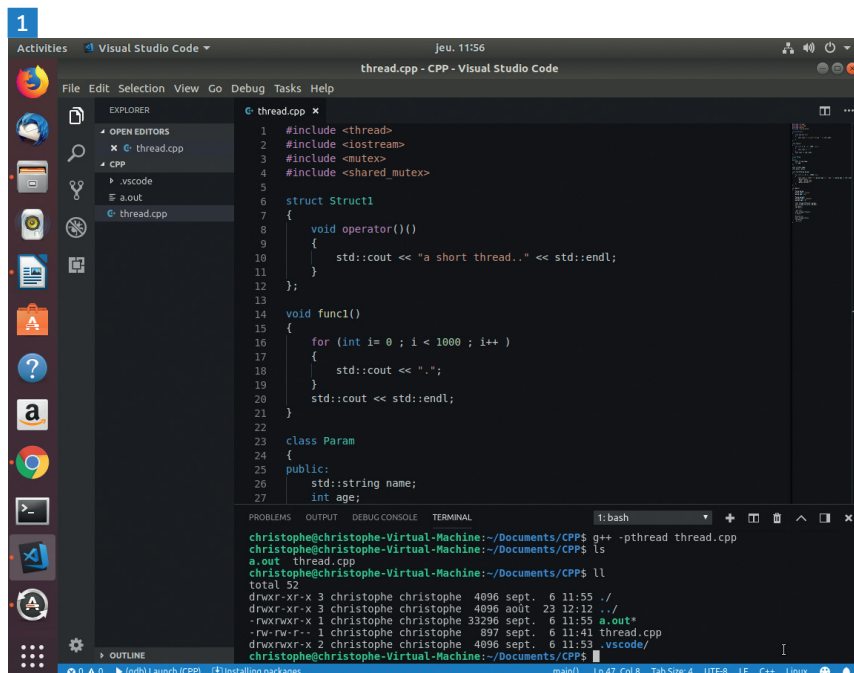
Pour créer un thread, on a deux solutions. Soit on utilise les API du système d'exploitation donc dans notre cas la librairie pthread, soit on utilise la STL et son fichier d'entête <thread>. L'avantage d'utiliser la STL est que le code est portable ou cross-plateforme. En utilisant directement pthread, mon code ne tournera que sur Linux... C'est la même chose sous Windows, si j'utilise la routine Win32 CreateThread, mon code est verrouillé pour Windows...

Mon premier thread

Le fichier d'entête <thread> de la STL permet de gérer les threads. Un thread est créé en instanciant la classe thread, et en lui passant en paramètre, soit une structure, une classe ou une fonction. La méthode join() permet d'attendre la fin d'exécution du thread.

```
#include <thread>
#include <iostream>

struct Struct1
{
    void operator()()
    {
    }
```



```

        std::cout << "a short thread.." << std::endl;
    }
};

void func1()
{
    for (int i=0; i < 1000; i++)
    {
        std::cout << ".";
    }
    std::cout << std::endl;
}

int main()
{
    std::thread t(func1);
    t.join();

    Struct1 s1;
    std::thread t2(s1);
    t2.join();
    return 0;
}

```

On remarquera que pour être utilisé avec une structure ou une classe, un thread utilise la redéfinition de l'opérateur () pour s'exécuter. La forme la plus simple est sinon de fournir une simple routine. Les threads s'exécutent dans le même espace mémoire et peuvent communiquer au travers d'objets partagés. De telles communications sont protégées par des verrous pour éviter les concurrences d'accès qui provoquent des plantages d'application car il y a corruption de la mémoire en cas d'accès concurrents sur la même donnée.

Compilation avec g++

La suite de compilation est celle de GCC et plus particulièrement g++ qui est le compilateur c++. Pour compiler un programme qui utilise les threads, il faut préciser l'option -pthread ; on utilise le standard C++11 via -std=c++11 :

```
$ g++ -pthread -std=c++11 thread.cpp -o thread
```

Cela produit un fichier exécutable thread que l'on peut lancer via ./thread.

Passage d'arguments

Pour pouvoir effectuer son job, un thread a besoin de paramètre(s). Il suffit de les passer en arguments au constructeur de la classe thread :

```

class Param
{
public:
    std::string name;
    int age;
};

```

```

void func3(Param param)
{
    std::cout << "Name:" << param.name << ", Age:" << param.age << std::endl;
}

int main()
{
    Param param;
    param.name = "Lisa";
    param.age = 12;
    std::thread t3(func3, param);
    t3.join();
}

```

Dans cet exemple, la routine func3 prend une classe Param en paramètre.

Retour de valeur

Il est possible pour un thread de retourner des éléments, une valeur, des champs, etc. Voir code complet sur le site.

Dans cet exemple, on passe l'adresse d'un double au constructeur de la structure en plus de la classe Param. On fait pointer ce membre sur l'adresse d'une variable membre et ensuite, le thread modifie la valeur membre. C'est tout !

Partage de données

Il est parfois nécessaire de partager des données. Dans ce cas, les accès aux données doivent être protégés. L'objet le plus simple pour y parvenir est le mutex : *mutual exclusion object*. Cet objet est généralement déclaré en global ou dans la classe et est utilisé pour protéger un bloc de données.

```

std::string _name;
std::mutex _mutex;

void func5(Param param)
{
    for (int i = 0; i < 100; i++)
    {
        std::cout << "Name:" << param.name << ", Age:"
        << param.age << std::endl;
        _mutex.lock();
        _name = param.name;
        _mutex.unlock();
    }
}

int main()
{
    Param param;
    param.name = "Lisa";
    param.age = 12;

    Param param2;
    param2.name = "Audrey";
}

```

```

param2.age = 8;

std::thread t5(func5, param);
std::thread t6(func5, param2);
t5.join();
t6.join();
}

```

La meilleure façon d'introduire une pause dans un thread est d'utiliser la routine `sleep_for()` elle redonne la main aux autres threads. Voici comment faire une attente de 250 millisecondes :

```
std::this_thread::sleep_for(std::chrono::milliseconds(250));
```

Il est aussi possible d'utiliser un lock plus léger qui est le `shared_mutex` :

```

std::shared_mutex _smutex;

void func6(Param param)
{
    for (int i = 0; i < 10000; i++)
    {
        _smutex.lock();
        _name = param.name;
        _smutex.unlock();

        _smutex.lock_shared();
        std::cout << "Name:" << _name << std::endl;
        _smutex.unlock_shared();
    }
}

int main()
{
    Param param;
    param.name = "Lisa";
    param.age = 12;

    Param param2;
    param2.name = "Audrey";
    param2.age = 8;

    std::thread t5(func6, param);
    std::thread t6(func6, param2);
    t5.join();
    t6.join();
}

```

Dans ce cas, on utilise la méthode `lock_shared()` pour l'accès en lecture et la méthode `lock()` pour l'accès en écriture.

Tâches : future et promise

Il existe un mécanisme de plus haut niveau que les threads et les mutex : ce sont les classes disponibles dans l'entête `<future>`. Lorsqu'un transfert de valeur entre deux tâches peut se faire sans verrous, on met en œuvre les futures et les promises, le tout

étant géré par des tâches plutôt que des threads. Quand une tâche veut passer une valeur à une autre tâche, elle positionne la valeur dans la promise ; la mécanique fait que la valeur est accessible en lecture dans la future. L'appel se fait de la sorte si nous avons une future `<X>` fx :

```
Xx = fx.get();
```

Exemple de future :

```

int MySum(int i, int j)
{
    return i + j;
}

void ft()
{
    std::packaged_task<int(int, int)> task(MySum); // wrap the function
    std::future<int> f1 = task.get_future(); // get a future
    std::thread t(std::move(task), 10, 20); // launch on a thread
    f1.wait();
    std::cout << "Done! result: " << f1.get() << std::endl;
    t.join();
}

```

Il est aussi possible de faire ça avec une promise en mettant la valeur via un `set_value` :

```

void ft3()
{
    // future from a promise
    std::promise<int> p;
    std::future<int> f3 = p.get_future();
    std::thread([&p] { p.set_value_at_thread_exit(9); }).detach();

    std::cout << "Waiting..." << std::flush;
    f3.wait();
    std::cout << "Done! result: " << f3.get() << std::endl;
}

```

Le plus simple restant l'utilisation de `async` :

```

void ft2()
{
    // future from an async()
    std::future<int> f2 = std::async(std::launch::async, [] { return 8; });
    f2.wait();
    std::cout << "Done! result: " << f2.get() << std::endl;
}

```

Conclusion

L'utilisation des threads n'est pas très compliquée. Il suffit de prendre garde à bien protéger les données partagées et de mettre en place les mutex qui vont bien. Pour une utilisation de plus haut-niveau, on notera que `async` peut-être facile à utiliser aussi. Le C++ est ISO et multiplateforme donc ce qui est valable sur Linux l'est aussi sur les autres plateformes comme Android, iOS, macOS ou Windows via la STL (Standard Template Library) du C++.